Lab Assignment 18.1

Task 1 – Weather API Integration

Connect to a Weather API to fetch current weather details for a given city.

Instructions:

- Prompt AI to generate Python code using the requests library.
- Handle errors: invalid city name, API key errors, and network failures.
- Display temperature, humidity, and weather description in a readable format.

Expected Output:

- A working Python script that fetches and displays weather data with proper error handling.

```python
"""
Weather fetcher -use requests to generate a weather report consisting of temperature, humidity, and weather conditions for a given location.
"""
import requests
def get_weather(location):
    #Get the latitude and longitude of a location using Open-Meta Geocoding API
    url = f"https://geocoding-api.open-meteo.com/v1/search?name={location}"
    response = requests.get(url)
    data = response.json()
    if data['results']:
        latitude = data['results'][0]['latitude']
        longitude = data['results'][0]['longitude']
        #Get the current weather for the given latitude and longitude using Open-Meteo Weather API
        weather_url = f"https://api.open-meteo.com/v1/forecast?latitude={latitude}&longitude={longitude}&current_weather=true"
        weather_response = requests.get(weather_url)
        weather_data = weather_response.json()
        if 'current_weather' in weather_data:
            return weather_data['current_weather']
        else:
            raise ValueError("Weather data not found")
    else:
        raise ValueError("Location not found")
def main():
    location = input("Enter a location: ")
    try:
        weather = get_weather(location)
        print(f"Current weather in {location}:")
        print(f"Temperature: {weather['temperature']}°C")
        print(f"Wind Speed: {weather['windspeed']} km/h")
        print(f"Weather Code: {weather['weathercode']}")
    except ValueError as e:
        print(e)
if __name__ == "__main__":
    main()
```

Task 2 – Currency Exchange Rates

Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.

Instructions:

- Use AI to generate API request code.
- Validate user input for amount and currency codes.
- Implement error handling for invalid responses and API downtime.

• Show conversion results clearly in tabular format.

Expected Output:

• A script that converts INR to multiple currencies with graceful error handling.

```python
"""Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP. using https://exchangerate-api.com/v4/latest/INR"""
import requests
def convert_currency(amount, from_currency, to_currency):
    #Convert currency using exchangerate-api.com API
    url = f"https://api.exchangerate-api.com/v4/latest/{from_currency}"
    response = requests.get(url)
    data = response.json()
    if 'rates' in data and to_currency in data['rates']:
        rate = data['rates'][to_currency]
        return amount * rate
    else:
        raise ValueError("Currency not found")
def main():
    amount = float(input("Enter the amount in INR: "))
    to_currency = input("Enter the target currency (USD, EUR, GBP): ").upper()
    try:
        converted_amount = convert_currency(amount, 'INR', to_currency)
        print(f"{amount} INR is equal to {converted_amount:.2f} {to_currency}")
    except ValueError as e:
        print(e)
if __name__ == "__main__":
    main()
```

Task 3 – GitHub Repository Info Fetcher

Connect to the GitHub API to fetch details of a repository (e.g., stars,forks, issues).

Instructions:

• Prompt AI to write a Python function to send GET requests to GitHub API.

• Handle rate limit errors, 404 (repo not found), and invalid input.

• Display repository name, description, stars, forks, and open issues.

Expected Output:

• A Python program that retrieves and displays GitHub repository information with robust error handling.

```python
"""write a Python function to send GET requests to GitHub API.Handle rate limit errors, 404 (repo not found), and invalid input.
Display repository name, description, stars, forks, and open issues and give one example of how to use the function to fetch details of a specific repository.
"""
import requests
def get_github_repo_details(owner, repo):
    #Get repository details from GitHub API
    url = f"https://api.github.com/repos/{owner}/{repo}"
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        return {
            'name': data['name'],
            'description': data['description'],
            'stars': data['stargazers_count'],
            'forks': data['forks_count'],
            'open_issues': data['open_issues_count']
        }
    elif response.status_code == 404:
        raise ValueError("Repository not found")
    elif response.status_code == 403:
        raise ValueError("Rate limit exceeded")
    else:
        raise ValueError("An error occurred")
def main():
    owner = input("Enter the repository owner: ")
    repo = input("Enter the repository name: ")
    try:
        details = get_github_repo_details(owner, repo)
        print(f"Repository Name: {details['name']}")
        print(f"Description: {details['description']}")
        print(f"Stars: {details['stars']}")
        print(f"Forks: {details['forks']}")
        print(f"Open Issues: {details['open_issues']}")
    except ValueError as e:
        print(e)
if __name__ == "__main__":
    main()
```

Task 4 – Real-Time Application: News Headlines Aggregator

Scenario: Build a News Aggregator using a free News API.

Requirements:

• Fetch top 5 headlines for a given category (sports, technology, health).

• Handle errors like invalid category, missing API key, and request failures.

• Display headlines in a user-friendly numbered list format.

• Include a retry mechanism if the first request fails.

Expected Output:

• A working Python script that retrieves and displays news headlines with proper error handling

```python
"""
Fetch top 5 headlines for a given category (sports, technology,health).also use newapi.org API to fetch news headlines.
Handle errors like invalid category, missing API key, and request failures.
Display headlines in a user-friendly numbered list format.
Include a retry mechanism if the first request fails.
"""
import requests
def fetch_headlines(category):
    #Fetch top 5 headlines for a given category using newapi.org API
    api_key = "YOUR_API_KEY"
    url = f"https://newsapi.org/v2/top-headlines?category={category}&apiKey={api_key}"
    try:
        response = requests.get(url)
        response.raise_for_status()  # Check for HTTP errors
        data = response.json()
        if data['status'] == 'ok' and data['totalResults'] > 0:
            return [article['title'] for article in data['articles'][:5]]
        else:
            raise ValueError("No headlines found for this category")
    except requests.exceptions.RequestException as e:
        print(f"Request failed: {e}")
        return None
def main():
    category = input("Enter a news category (sports, technology, health): ").lower()
    if category not in ['sports', 'technology', 'health']:
        print("Invalid category. Please choose from sports, technology, or health.")
        return
    headlines = fetch_headlines(category)
    if headlines:
        print(f"Top 5 headlines in {category}:")
        for i, headline in enumerate(headlines, 1):
            print(f"{i}. {headline}")
```